Lab 12

Introduction

In the Producer-Consumer problem, one or more producers generate data and place it into a shared buffer, while one or more consumers remove data from the buffer. Proper synchronization is required to avoid:

- **Race conditions:** Multiple threads modifying the buffer simultaneously.
- **Buffer underflow:** Consumers trying to consume from an empty buffer.
- **Buffer overflow:** Producers trying to add to a full buffer.

Synchronization Mechanisms

To solve this problem in Python, we use:

1. **Mutex (Lock):** Ensures exclusive access to the shared buffer so that only one thread can modify it at a time.
2. **Empty (Semaphore):** Counts the number of empty slots in the buffer. A producer must wait if no empty slots exist.
3. **Full (Semaphore):** Counts the number of filled slots in the buffer. A consumer must wait if no items exist.

Implementation Logic

Producer Logic

1. Wait for an empty slot (`empty.acquire()`).
2. Acquire mutex (`mutex.acquire()`) to safely access the buffer.
3. Add the item to the buffer.
4. Release mutex (`mutex.release()`).
5. Signal that a new item is available (`full.release()`).

Consumer Logic

1. Wait for a filled slot (`full.acquire()`).
2. Acquire mutex (`mutex.acquire()`) to safely access the buffer.
3. Remove an item from the buffer.
4. Release mutex (`mutex.release()`).
5. Signal that a new empty slot is available (`empty.release()`).

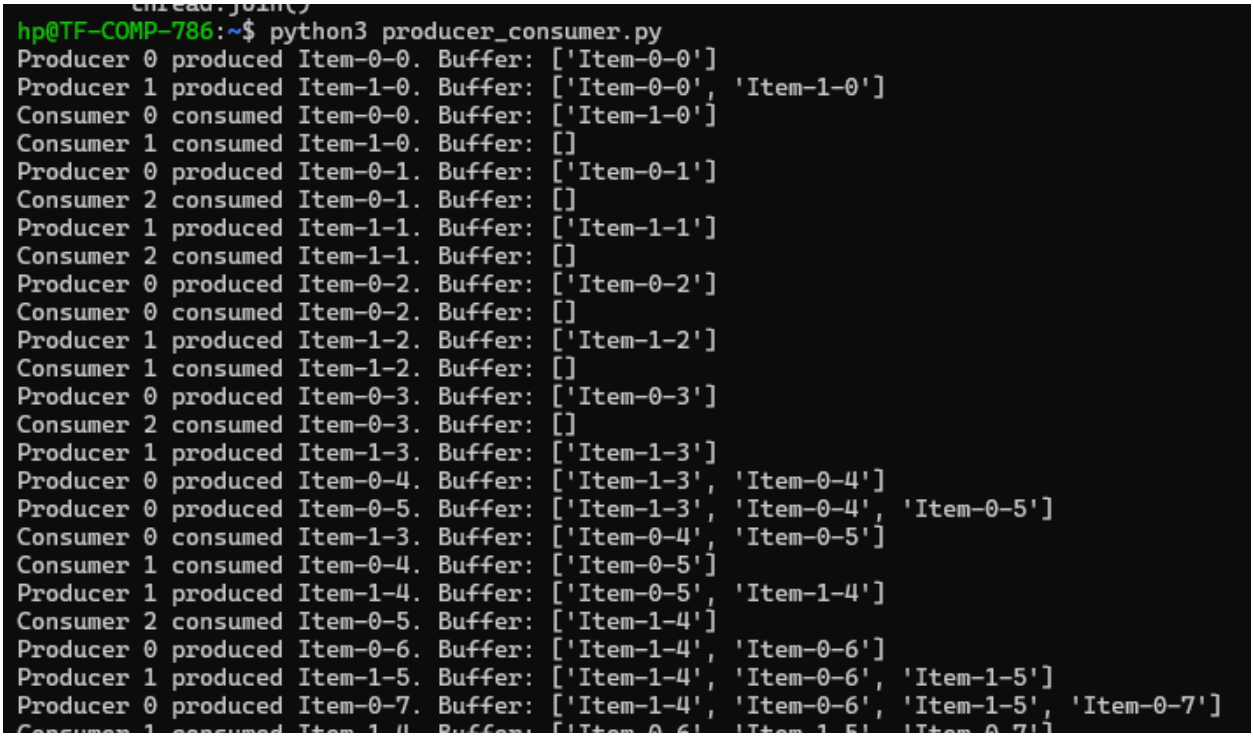
```
hp@TF-COMP-786:~$ cat > producer_consumer.py
# producer_consumer.py
import threading
import time
import random
from collections import deque
BUFFER_SIZE = 5
PRODUCER_COUNT = 2
CONSUMER_COUNT = 3

buffer = deque(maxlen=BUFFER_SIZE)
empty = threading.Semaphore(BUFFER_SIZE)  # Tracks empty slots
full = threading.Semaphore(0)             # Tracks filled slots
mutex = threading.Lock()                   # Ensures mutual exclusion

def producer(producer_id):
    global buffer
    item_counter = 0
    while True:
        item = f"Item-{producer_id}-{item_counter}"
        empty.acquire()
        mutex.acquire()
        buffer.append(item)
        print(f"Producer {producer_id} produced {item}. Buffer: {list(buffer)}")
        mutex.release()
        full.release()
        item_counter += 1
        time.sleep(random.uniform(0.5, 2))

# --- Consumer Thread ---
def consumer(consumer_id):
    global buffer
    while True:
        full.acquire()
        mutex.acquire()
        item = buffer.popleft()
        print(f"Consumer {consumer_id} consumed {item}. Buffer: {list(buffer)}")
        mutex.release()
        empty.release()
        time.sleep(random.uniform(1, 3))

# --- Main Application ---
if __name__ == "__main__":
    threads = []
    for i in range(PRODUCER_COUNT):
        thread = threading.Thread(target=producer, args=(i,))
        threads.append(thread)
        thread.start()
    for i in range(CONSUMER_COUNT):
        thread = threading.Thread(target=consumer, args=(i,))
        threads.append(thread)
        thread.start()
    for thread in threads:
        thread.join()
```



Role of mutex

The mutex ensures exclusive access to the shared buffer. By allowing only one thread at a time to modify the buffer, it prevents race conditions where two producers or consumers could simultaneously change the buffer contents, potentially causing items to be lost, overwritten, or the buffer to become inconsistent. If removed, two threads could modify the buffer at the same time, leading to errors like overwriting items or losing data.

Role of empty

The empty semaphore keeps track of the number of available slots in the buffer. It ensures that producers do not attempt to add items when the buffer is full. Without the empty semaphore, a producer could insert an item into a full buffer, which would lead to buffer overflow or dropped data.

Role of full

The full semaphore monitors the number of filled slots in the buffer. It ensures that consumers do not try to remove items when the buffer is empty. Without the full semaphore, a consumer could attempt to consume from an empty buffer, resulting in runtime errors, exceptions, or program crashes. If removed**,** a consumer might try to pop from an empty buffer, causing exceptions or crashes.

Mutex protects shared resources. Empty and Full semaphores synchronize the producer-consumer relationship. All three primitives are essential to safely and efficiently implement the producer-consumer problem.